



FUN WITH PYTHON: GENERATORS & ITERATORS

Learning how Python helps
us run through data



July 30, 2026



WHAT IS AN ITERATOR?

An iterator is like a magical bookmark that helps us go through a list one item at a time.

- It helps computers remember where they are in a list.
- Think of it like turning pages in a book – you only see one page at a time!
- Iterators make it easy to visit every item without getting lost.

In our coding adventure, we'll learn how to use these helpful tools to work with lists of numbers, words, and more!

WHY DO WE USE ITERATORS?

Iterators are super helpful tools that make working with data much easier and more organized. They help us focus on one thing at a time instead of getting overwhelmed by big lists of information.



To look at one thing at a time

To not get lost in big lists

To save time and energy

HOW DOES ITERATION WORK IN PYTHON?

In Python, we use loops to go through each item in a list. Think of it like reading a story one page at a time!

```
fruits = ['apple', 'banana', 'mango']  
for fruit in fruits:  
    print(fruit)
```

This loop shows each fruit in the list, one by one!

LOOPING THROUGH LISTS STEP-BY-STEP

- Watch how the loop goes through each fruit one at a time!
- Step 1: The loop starts with fruit = 'apple' and prints it first.
- Step 2: Now the loop moves to fruit = 'banana' and prints it second.
- Step 3: Finally, the loop reaches fruit = 'mango' and prints it last.
- The loop keeps going until there are no more items left in the list!

WHAT IS A GENERATOR?

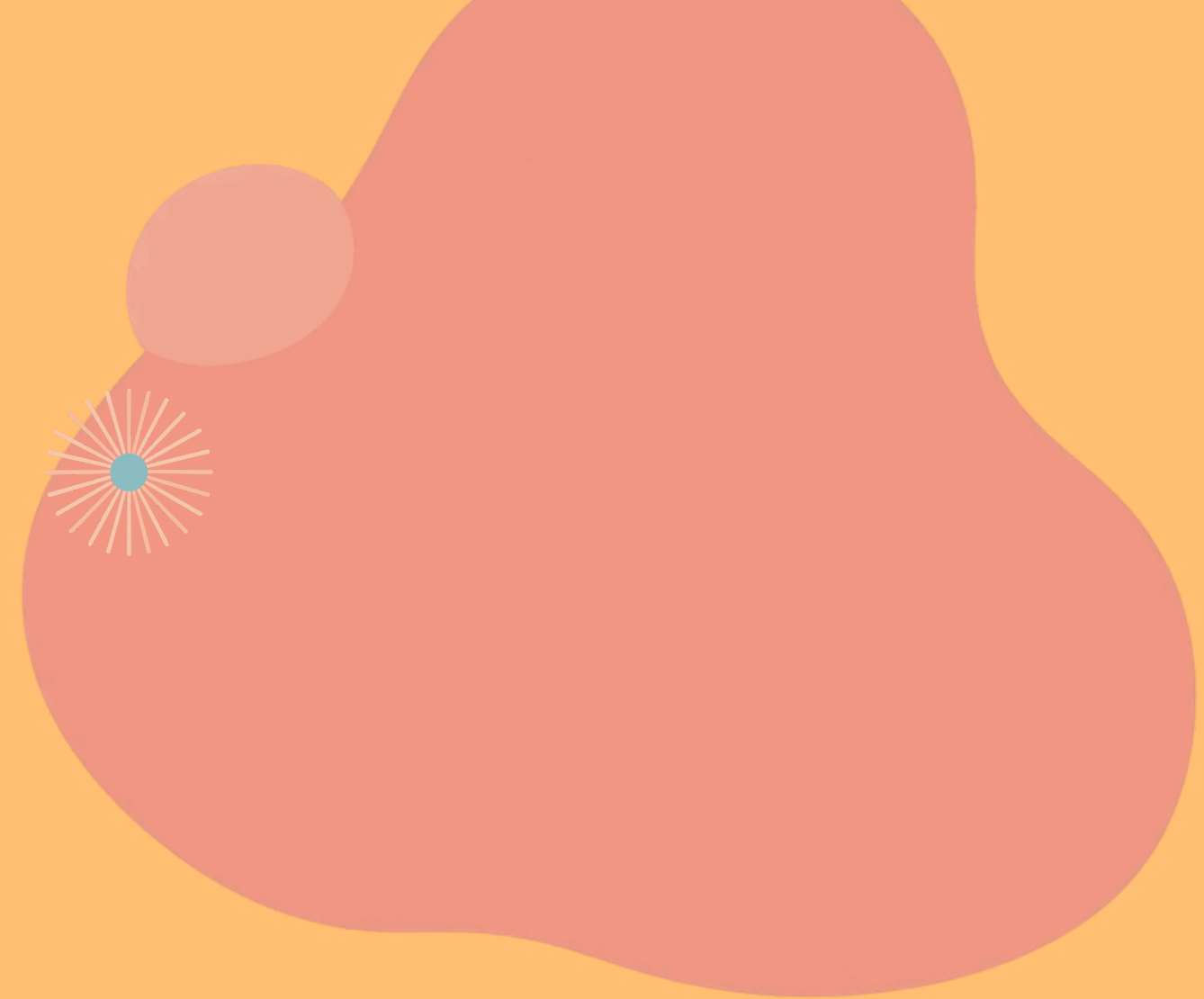
- A generator is like a magic machine that creates items one at a time when you ask for them.
- Think of a toy factory! When you press a button, it makes just one toy. Press again, and it makes another. It doesn't make all the toys at once – that would take up too much space!
- This is cool because:
 - It saves space because it doesn't make all items at once.
 - It's efficient – only creates what you need, when you need it.
 - It's perfect for big lists of things!

HOW GENERATORS WORK IN PYTHON

- Here's a simple generator that counts to three:
- `def count_up_to_three():`
 - `yield 1`
 - `yield 2`
 - `yield 3`
 -
- `for number in count_up_to_three():`
 - `print(number)`
- The 'yield' word is special! It sends one item out at a time, pausing the function and waiting for the next request. This is how generators work in Python!

ITERATORS VS GENERATORS

Understanding the difference between iterators and generators helps us write smarter, more efficient Python code.
Think of them as two different ways to go through data!



Iterators:
Go through items one by one



Iterators:
Can be created from lists



Generators:
Make items only when needed



Generators:
Use 'yield' to give items

PRACTICAL EXAMPLE 1: USING AN ITERATOR

Code Example:

```
my_list = [10, 20, 30]  
it = iter(my_list)  
print(next(it)) # prints 10  
print(next(it)) # prints 20
```

We can ask an iterator for the next item anytime using 'next()'. It's like asking the magical bookmark to show us the current page!

Each time we call next(), we move to the next item in our list.

PRACTICAL EXAMPLE 2: A GENERATOR IN ACTION

Here is a simple generator that creates numbers one at a time:

```
def my_generator():  
    for i in range(1, 4):  
        yield i
```

```
for num in my_generator():  
    print(num)
```

The generator gives numbers 1 to 3, one at a time. Each time we call it, it produces the next number using 'yield'.

This is super efficient because Python only creates the numbers we actually need, not all of them at once!



HANDS-ON ACTIVITY: FIND THE NEXT ITEM!



If we have a list of colors ['red', 'green', 'blue'], what will 'next()' show first?

✨ Try this challenge:

Write code to get each color in order using next()!

Think about it: When you call next() on an iterator, it gives you the very first item in the list. So what's the first item in our color list?



HANDS-ON ACTIVITY 2

Create Your Own Generator!

Make a generator that gives your favorite animals one by one.

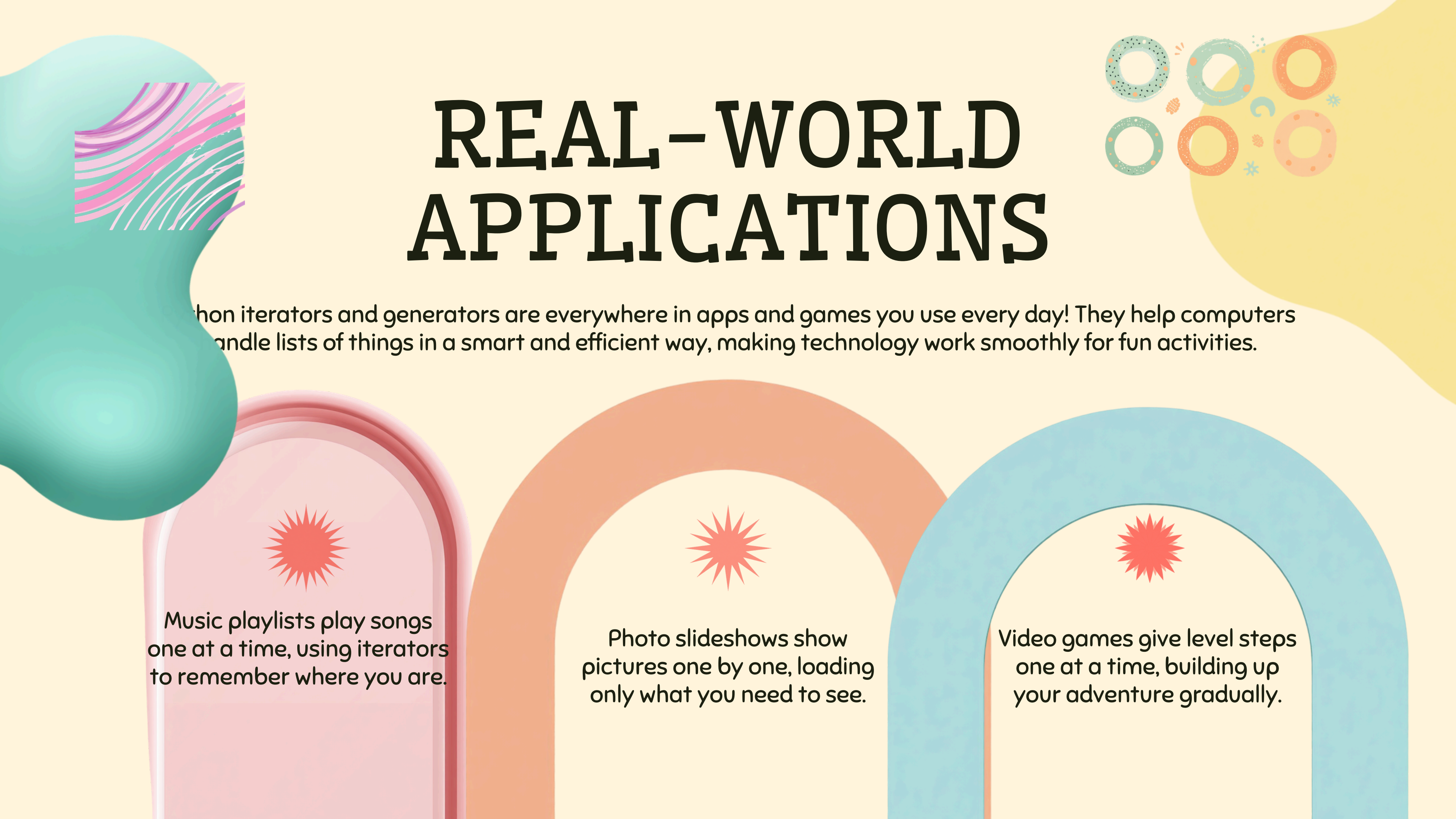
Hint: Use 'yield' for each animal!

Try this example:

```
def my_animals():  
    yield 'dog'  
    yield 'cat'  
    yield 'elephant'  
    yield 'tiger'  
for animal in my_animals():  
    print(animal)
```

Challenge: Add your own favorite animals to the generator!





REAL-WORLD APPLICATIONS

When iterators and generators are everywhere in apps and games you use every day! They help computers handle lists of things in a smart and efficient way, making technology work smoothly for fun activities.



Music playlists play songs one at a time, using iterators to remember where you are.



Photo slideshows show pictures one by one, loading only what you need to see.



Video games give level steps one at a time, building up your adventure gradually.

WHAT WE LEARNED TODAY

Great job exploring Python generators and iterators! Let's review the key things we discovered about how these powerful tools help us work with data efficiently.

Iterators help us go through items one by one.

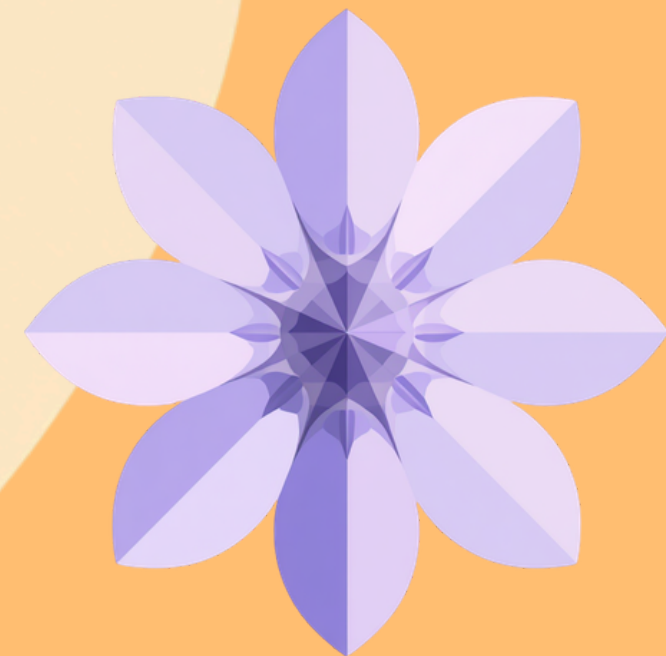
Loops use iterators to repeat actions.

Generators create items only when we ask for them.

Using these helps us write smart and fast programs.

KEEP EXPLORING PYTHON!

Now that you know about iterators and generators, try making your own! Practice on websites like Code.org, Scratch, or Python.org for kids. Download a kid-friendly Python app and start building programs that use these cool features. Remember: every expert coder started as a beginner. Have fun experimenting with loops, lists, and yield statements. The more you practice, the more creative your code will become!





**THANKS FOR
LEARNING
WITH US!**

